Laboratorium Ilmu Rekayasa dan Komputasi

TEKNIK INFORMATIKA

STEI ITB

IF2224 - Formal Language and Automata Theory

Arion Compiler

Milestone 1 - Lexical Analysis

IF2224

Milestone 1
Lexical Analysis

Release : **Kamis, 12 Maret 2026**

Deadline : **Kamis, 2 April 2026 pukul 23.59 WIB**

Revisi : -

Daftar Revisi

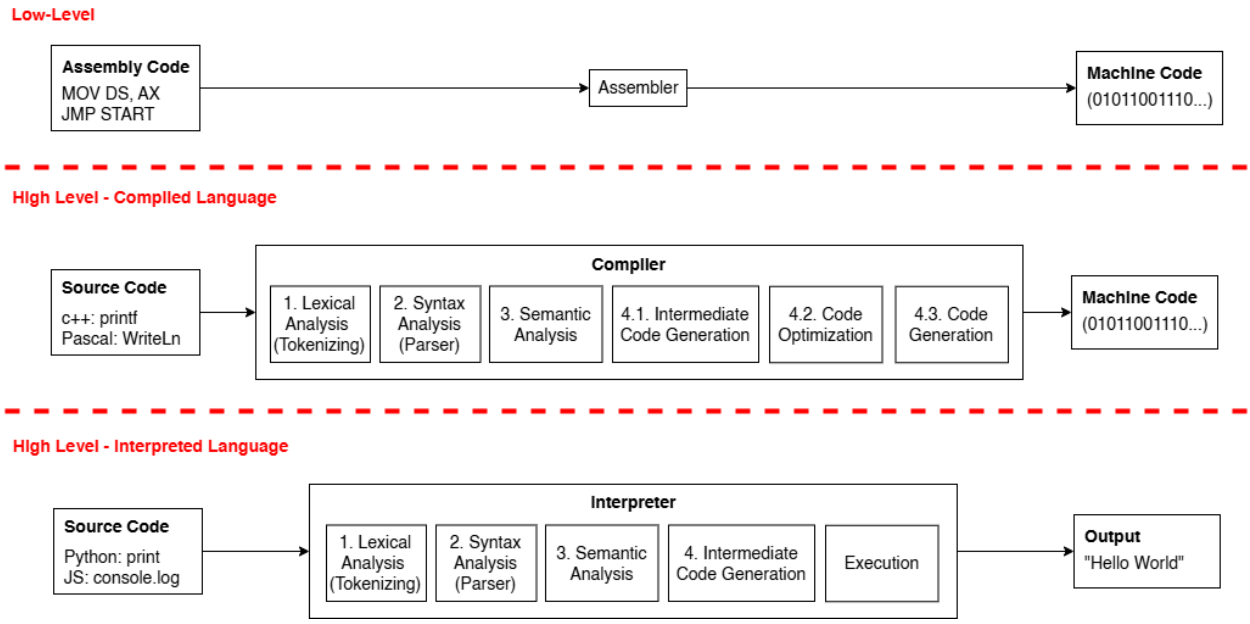
[DD-MM-YY] Revisi X. Isi Revisi

Daftar Isi

Daftar Revisi **3**
Daftar Isi **4**
I. Pendahuluan Tugas Besar **5**
II. Teori Singkat **7**
III. Spesifikasi Tugas Besar **8**
A. Spesifikasi Program 8
B. Daftar Token Bahasa Pemrograman Arion 9
C. Contoh Input/Output 12
D. Diagram Transisi DFA 13
E. Laporan 14
IV. Teknis Pengerjaan **15**
VI. Pengumpulan **16**
V. Penilaian **17**
Referensi **19**
Lampiran **20**

I. Pendahuluan Tugas Besar

Apa itu **Bahasa Pemrograman**? Bahasa pemrograman adalah cara agar kita dapat menuliskan program komputer tanpa perlu menghadapi sulitnya menulis dalam bahasa mesin. Bahasa mesin (biner 1 dan 0) adalah satu-satunya bahasa yang dapat dipahami dan dijalankan oleh CPU, namun sangat sulit dibaca oleh manusia. Oleh karena itu, bahasa pemrograman sering kali dibedakan dengan istilah low-level atau high-level. High-level artinya bahasa lebih mendekati bahasa manusia dan sebaliknya low-level lebih mendekati bahasa mesin. Semakin mendekati bahasa manusia, maka akan semakin banyak proses yang diperlukan untuk mengubahnya menjadi kode mesin.



Gambar 1. Tipe-tipe bahasa pemrograman
(Sumber: asisten)

Terdapat dua macam cara mengubah/memproses bahasa pemrograman menjadi machine code, yaitu **compiler** dan **interpreter**. Compiler adalah program yang menerjemahkan seluruh source code menjadi kode mesin (dalam bentuk file e.g. Object File) sebelum program dijalankan. Compiler secara umum terdiri dari 4 tahapan, mulai dari Lexical Analysis, Syntax Analysis, Semantic Analysis hingga Intermediate Code Generation.

Di sisi lain adalah Interpreter yang memproses source code baris per baris secara langsung. Interpreter membaca, menerjemahkan, dan langsung mengeksekusi kode mesin baris per baris sehingga jika terdapat error pada baris ke 10, program akan tetap berjalan normal untuk 9 baris awal, dibandingkan dengan compiler yang akan gagal dijalankan jika terdeteksi terdapat error di dalam source code. Namun dibalik itu, interpreter sebenarnya memiliki tahapan yang sama seperti compiler.

Pada Tugas Besar Teori Bahasa Formal dan Automata, Anda diminta untuk membuat **Interpreter** untuk bahasa pemrograman **Arion**, suatu bahasa pemrograman baru yang akan Anda kembangkan. Tugas besar ini dibagi menjadi **4 Milestone**, yaitu **Lexical Analysis, Syntax Analysis, Semantic Analysis, Intermediate Code Generation** dan **Interpreter.** Setiap Milestone akan memiliki pengumpulannya dan penilaiannya tersendiri.

Pada milestone pertama, Anda diminta untuk membuat tahapan Lexical Analysis dari suatu interpreter.

II. Teori Singkat

Analisis leksikal (lexical analysis) adalah tahap pertama dalam proses kompilasi atau interpretasi bahasa pemrograman. Pada tahap ini, *interpreter* membaca kode sumber mentah yang pada dasarnya hanyalah rangkaian karakter dan mengubahnya menjadi rangkaian satuan makna yang disebut token. **Token merupakan unit terkecil yang memiliki arti dalam sebuah program**, misalnya kata kunci (keyword), nama variabel (identifier), operator, angka, atau tanda baca.

Tujuan utama dari analisis leksikal adalah untuk mempermudah tahap berikutnya, yaitu parsing. Alih-alih berurusan langsung dengan karakter mentah, tahap parsing bekerja dengan kumpulan token yang sudah terstruktur, sehingga lebih mudah memahami susunan logis dari program. Pada proses ini, analisis leksikal juga membuang bagian yang tidak diperlukan seperti spasi, tab, dan komentar, serta dapat mendeteksi kesalahan seperti simbol yang tidak dikenal sejak awal.

Proses ini dilakukan oleh komponen yang disebut lexer. **Lexer membaca kode sumber dari kiri ke kanan dan menggunakan pola tertentu untuk mengenali jenis token yang berbeda-beda**. Ketika menemukan urutan karakter yang cocok dengan salah satu pola tersebut, lexer membuat sebuah token yang terdiri dari jenis token dan nilai token. Contoh hasil token:

- IDENT (a)
- BECOMES
- INTCON (5)

Agar proses pembacaan tersebut dapat dijalankan secara otomatis oleh komputer, mereka harus diimplementasikan ke dalam bentuk Finite Automata, khususnya **Deterministic Finite Automata (DFA)**.

III. Spesifikasi Tugas Besar

Pada milestone 1, Anda diminta untuk membuat **lexer atau lexical analyzer** dalam **C/C++** yang menggunakan **Deterministic Finite Automata (DFA)** untuk menghasilkan **token** dari source code bahasa pemrograman Arion.

Deliverables:

- Source Code program,
- Diagram Transisi DFA, dan
- Laporan.

A. Spesifikasi Program

Dalam milestone ini, Anda diminta untuk membuat **program Lexer** dalam bahasa pemrograman **C/C++ GNU**. Anda **DILARANG menggunakan library atau tools** apapun yang dapat membantu menghasilkan lexical analyzer. Anda wajib menggunakan **MAKEFILE** untuk mempermudah pengembangan dalam bahasa C/C++.

Program Lexer harus berfungsi selayaknya suatu **DFA**. Program Lexer **HANYA BOLEH** membaca **satu demi satu karakter** dari input source code dan memprosesnya sesuai dengan aturan pada DFA. Program harus dapat menerima **input source code** dalam bentuk **.txt**. Program kemudian memproses source code dengan **aturan DFA** dan mengeluarkan **output** **daftar token** dalam format **.txt**.

Masukan	Kode Sumber (.txt)
Keluaran	Daftar Token (.txt)
Otomata	Deterministic finite automata (DFA)

Catatan Penting: Untuk mempermudah pengerjaan pada milestone-milestone selanjutnya, khususnya pada milestone 4 (pembuatan interpreter), SANGAT DISARANKAN agar program dikembangkan secara modular (terpisah antara komponen Lexer, Semantic, Syntax, dan Intermediate Code Generation). Hal ini supaya komponen dapat dimodifikasi sesuai keperluan tanpa mempengaruhi komponen lainnya, serta dapat dengan mudah dijalankan dalam satu pipeline: lexer - syntax - semantic - intermediate code generation.

B. Daftar Token Bahasa Pemrograman Arion

Lexer bertujuan untuk mengubah source code dari kumpulan karakter mentah menjadi unit-unit bermakna yang disebut **token**. Setiap token memiliki **jenis (type)** dan **nilai (value)** yang akan digunakan untuk tahap selanjutnya dalam proses kompilasi. Lexer diharapkan dapat menerima source code dan mengeluarkan daftar token yang dihasilkan.

Berikut merupakan daftar token yang harus dapat dibaca oleh lexical analyzer:

Daftar Token			
No	Token	Definisi & Keterangan	Contoh Pemakaian
1	**intcon**	Konstanta integer	1, 3, 48
2	**realcon**	Konstanta bilangan riil	3.14, 26.7
3	**charcon**	Konstanta karakter singular, diapit dengan tanda petik tunggal (')	'j', 'k', 't'
4	**string**	Sekuens karakter, diapit dengan tanda petik tunggal (')	'IRK', 'TBFO'
5	**notsy**	Operator logika NOT (negasi)	NOT
6	**plus**	Operator aritmatika pertambahan	+
7	**minus**	Operator aritmatika pengurangan	-
8	**times**	Operator aritmatika perkalian	*
9	**idiv**	Operator aritmatika pembagian Integer	div
10	**rdiv**	Operator aritmatika pembagian Riil	/
11	**imod**	Operator aritmatika modulo	MOD
12	**andsy**	Operator logika AND	AND
13	**orsy**	Operator logika OR	OR

14	**eql**	equal	==
15	**neq**	Not equal	<>
16	**gtr**	Greater than	>
17	**geq**	Greater than or equal	>=
18	**lss**	Less than	<
19	**leq**	Less than or equal	<=
20	**lparent**	Left parentheses	(
21	**rparent**	Right parantheses	)
22	**lbrack**	Kurung siku kiri	[
23	**rbrack**	Kurung siku kanan	]
24	**comma**	-	,
25	**semicolon**	-	;
26	**period**	-	.
27	**colon**	-	:
28	**becomes**	Assignment operator	:=
29	**constsy**	Deklarasi konstanta	const
30	**typesy**	Deklarasi tipe data	type
31	**varsy**	Deklarasi variabel	var
32	**functionsy**	Deklarasi fungsi	function
33	**proceduresy**	Deklarasi prosedur	procedure
34	**arraysy**	Deklarasi array	array
35	**recordsy**	Deklarasi record	record
36	**programsy**	Deklarasi program	program
37	**ident**	Nama yang menunjukkan	x, PI, MyInt,

		konstanta, tipe, variabel, prosedur, dan fungsi. Nama tersebut harus diawali dengan huruf, yang dapat diikuti oleh kombinasi huruf dan angka apa pun, dan berjumlah berapa pun. Bersifat case-insensitive	Integer, Real, Char, String h4g, identIniSangatPanjang24tetapiTetap Valid
38	**beginsy**	-	begin
39	**ifsy**	-	if
40	**casesy**	-	case
41	**repeatsy**	-	repeat
42	**whilesy**	-	while
43	**forsy**	-	for
44	**endsy**	-	end
45	**elsesy**	-	else
46	**untilsy**	-	until
47	**ofsy**	-	of
48	**dosy**	-	do
49	**tosy**	-	to
50	**downtosy**	-	downto
51	**thensy**	-	then
52	**comment**	Dimulai dengan { atau (* (bukan di dalam tanda kutip) dan diakhiri dengan } atau *).	{ Ini komentar }, (* In komentar *)

C. Contoh Input/Output

Anda wajib membuat **sebuah DFA** yang dapat menentukan pola karakter dan menghasilkan token. Berikut adalah contoh format Input/Output dari program.

Masukan (input)

```
program Hello;

var
  a, b: integer;

begin
  a := 5;
  b := a + 10;
  writeln('Result = ', b);
end.
```

Contoh visualisasi pemrosesan dalam program (State DFA belum tentu benar)

```
p => State 1
r => State 1
.
m => State 2 => Gotten: program
.
H => State 3
e
.
; => char => State 0 => Gotten: ident(Hello) & semicolon
.
.
.
```

Keluaran (output)

```
programsy
ident (Hello)
semicolon

varsy
ident (a)
comma
ident (b)
colon
ident (integer)
```

```
semicolon

beginsy
ident (a)
becomes
intcon (5)
semicolon
ident (b)
becomes
ident (a)
plus
intcon (10)
semicolon
ident (writeln)
lparent
string ('Result =')
comma
ident (b)
rparent
semicolon

endsy
period
```

D. Diagram Transisi DFA

Selain program, Anda wajib membuat **diagram transisi DFA** sesuai dengan **DFA** yang dibuat. Diagram wajib dibuat pada workspace berbasis web, seperti **draw io** (untuk workspace lain silahkan ditanyakan terlebih dahulu pada QNA). Pastikan pembuatan diagram transisi sesuai dengan aturan yang dipelajari di salinda perkuliahan.

DILARANG menggunakan library atau tools yang mengubah aturan DFA dalam program menjadi gambar diagram secara otomatis. Diagram transisi DFA harus dibuat secara manual. Hal ini dilakukan untuk memastikan bahwa mahasiswa memahami aturan DFA yang dibuatnya.

E . Laporan

Anda wajib membuat laporan yang setidaknya berisi:

- Cover dengan foto kelompok menggantikan logo ITB.
- Teori Singkat
  - Jelaskan teori-teori atau konsep yang digunakan (DFD, Lexer, Token, dsb)
- Perancangan & Implementasi
  - Cantumkan dan beri penjelasan diagram transisi DFA yang dibuat dan digunakan.
  - Jelaskan implementasi program, terutama yang berhubungan dengan Lexical Analyzer dan DFA.
- Pengujian
  - Setidaknya 5 pengujian kasus unik.
  - Setiap pengujian harus menunjukan hasil input/output program.
  - Bukti berupa screenshot input/output program.
- Kesimpulan dan Saran
- Lampiran berupa:
  - Link Repository Github.
  - Link Workspace Diagram Transisi DFA
  - Pembagian Tugas (menunjukan persentase kontribusi).
- Referensi

IV. Teknis Pengerjaan

- Tugas Besar ini dikerjakan secara **berkelompok 4-5 orang** dan **tidak diperbolehkan lintas kelas**.
- Setiap kelompok **WAJIB** mengisi sheets pembagian kelompok. Jika melewati deadline dan Anda tidak mengisi, maka akan dialokasikan secara acak.
    - Deadline Pengisian adalah **Kamis, 26 Maret 2026 pukul 23.59 WIB.**

- Buatlah **Repository Github** dengan format **[KODE]-Tubes-IF2224-2026**, dengan kode adalah Kode Kelompok sesuai sheets pembagian kelompok.
- Repository setidaknya mengandung:
    - Folder **src** untuk menyimpan seluruh **source code**
    - Folder **doc** untuk menyimpan **laporan**
    - Folder **test** untuk menyimpan **semua input/output** yang dilakukan pada bagian pengujian laporan. Untuk membedakan dengan milestone lain, maka file dipisahkan ke dalam folder untuk setiap milestone. Sebagai contoh: **`test/milestone-1/output-1.txt`**.
    - **README** yang setidaknya mengandung:
        - Identitas Kelompok
        - Deskripsi Program
        - Requirements
        - Cara Instalasi dan Penggunaan Program
        - Pembagian Tugas

- Jika ada pertanyaan lebih lanjut, silahkan ditanyakan pada QNA.

VI. Pengumpulan

- **Pengumpulan bersifat Hard Deadline**

- Pada setiap Milestone, Anda **WAJIB** membuat **Release.**
    - Gunakan format tag berupa **v0.X.Y** dengan **X** adalah **nomor milestone** dan **Y** adalah **nomor pengumpulan**.
    - Sebagai contoh, untuk milestone 1, release pertama, dan pengumpulan pertama adalah **v0.1.1**. Untuk pengumpulan selanjutnya karena revisi, gunakan tag **v0.1.2** dan seterusnya sesuai nomor pengumpulan. Untuk milestone selanjutnya, pengumpulan dimulai kembali dari satu yaitu **v0.2.1**, **V0.2.2**, dan seterusnya.

- **Laporan** dikumpulkan dengan cara **meletakkan** hasil **PDF** dalam **folder doc** dengan format **Laporan-X-[KODE].pdf**, dimana **X** adalah **nomor milestone.** Pastikan isi laporan sudah sesuai dengan **teknis pengerjaan**.

- Link Pengumpulan: **Pengumpulan Milestone**
- Deadline milestone: **Kamis, 02 April 2026 pukul 23.59 WIB**

V. Penilaian

Program (70 poin)			
No	Komponen	Poin	Syarat Poin Maksimal
1.	Kelengkapan token	10	Program mampu mengenali seluruh jenis token yang ditentukan dalam spesifikasi
2.	Kesesuaian klasifikasi token	10	Setiap token diidentifikasi dan diklasifikasikan dengan benar sesuai kategori
3.	Struktur/modularitas program	8	Kode ditulis dengan struktur modular menggunakan fungsi atau kelas yang terpisah berdasarkan tanggung jawab. Alur eksekusi program mudah diikuti dan tidak redundan
4.	Error handling	5	Program dapat menangani error dengan baik (pesan informatif dan error tidak menyebabkan crash)
5.	Penerimaan input	3	Program dapat membaca file input (.txt)
6.	Penghasilan output	2	Program menghasilkan output daftar token secara lengkap dan terformat dengan jelas ke terminal
7.	Dokumentasi/komentar	5	Kode diberi komentar yang informatif dan tidak berlebihan
8.	Kesesuaian DFA	12	Program berjalan selayaknya suatu DFA dan Source Code program sesuai dengan diagram transisi DFA
9.	Pengujian	15	Program berhasil lolos seluruh kasus uji

Laporan & Diagram (30 poin)			
No	Komponen	Poin	Syarat Poin Maksimal
1.	Penjelasan konsep	10	Penjelasan mengenai konsep lexer yang mencakup pengertian, fungsi, dan peran lexer dalam proses kompilasi, serta kaitannya dengan token dan DFA.
2.	Perancangan program	5	Penjelasan mengenai rancangan program yang mencakup teknologi yang digunakan, struktur program, fungsi/kelas utama, dan alur kerja program. Terdapat pula justifikasi atas pilihan implementasi
3.	Hasil implementasi	2	Dokumentasi mengenai hasil implementasi, dapat berupa tangkapan layar dengan penjelasan
4.	Test case	3	Dokumentasi mengenai pengujian mandiri (minimal 5 dan mencakup berbagai kasus)
5.	Diagram Transisi DFA	10	Diagram Transisi DFA digambar dengan benar dan lengkap

Referensi

"Compiler Design - Lexical Analysis"
https://www.tutorialspoint.com/compiler_design/compiler_design_lexical_analysis.htm

"Introduction to Automata Theory, Languages, and Computation 3rd Edition"
https://cdn-edunex.itb.ac.id/29161-Formal-Language-Theory-and-Automata/1629640939613_Introduction-to-Automata-Theory,-Languages,-and-Computation-Edition-3.pdf

"Compilers - Principles, Techniques, and Tools (2006)"
https://repository.unikom.ac.id/48769/1/Compilers%20-%20Principles,%20Techniques,%20and%20Tools%20(2006).pdf

"An Introduction to Makefiles"
https://www.gnu.org/software/make/manual/html_node/Introduction.html

Lampiran

Laman Pembagian Kelompok: Pembagian Kelompok - Tubes IF2224 TBFO
Laman QnA Tugas Besar: QNA - Tubes IF2224 TBFO
From Pengumpulan: Form Pengumpulan - Tubes IF2224 TBFO